MDM Address Validation Pipeline

Group: 5010 - 5

Team:

1. Aashlesha Rajput
2. Rachietaa Ramesh Rao
3. Pablo Garces
4. David Caballero
5. Purva Jagtap

Github: https://github.com/PabloG55/mdm-pipeline-5

A. The Problem

Honeywell maintains a Customer Master Data (MDM) system with about 5 million customer records. They use Dun & Bradstreet, a well-known third-party data enrichment service, to fill in details like the company's legal name, parent company, industry codes, website, and so on. For most records this works. But about 34,000 records consistently fail to enrich.

Why?

Because the source data is bad. Some records have addresses with missing street numbers, others have placeholder text where the street should be, some have the company name written as a code like "TLSI" or "RMWB" instead of the actual name, some are in Chinese or Spanish or Russian, and some have encoding corruption from when the data was imported (text like "Málaga" turned into "MÃ¡laga").

Without enrichment, those 34,000 records are essentially useless to downstream business processes. You can't bill them properly, you can't analyse them, you can't roll up their revenue under their parent company. The goal of this project is to recover them -to take that messy, multilingual, half-broken data and turn it into clean, enriched master records.

> We are building an automated pipeline that takes 34,000 broken customer records that D&B couldn't fix, and uses a combination of mapping APIs and AI to figure out who the company really is, where they actually are, and what they do - so the records can be rescued.

B. How the Pipeline Works - 4 Stages

Every record flows through four stages. Each stage either fixes the record, validates it, or hands it off to the next stage with extra information. The pipeline runs 15 records in parallel at a time and processes the full 36,230 records in about 4.5 hours.

1. Stage 1 - Pre-Processing (cleanup and translation)

This is where we take the raw mess and make it readable. The pipeline does five things here:

- **Splits the address** into street, city, state, country, zip, the source data uses a special separator "|#|#" to mark where one field ends and the next begins.
- **Fixes mojibake:**
  This is encoding corruption, where UTF-8 text was misread as Latin-1 and the special characters got mangled. The pipeline detects the canonical mojibake pattern, decodes it backwards, and restores the original text. In this run, 7,720 records (21.3% of all records) had mojibake that was automatically fixed, these would have been completely unreadable otherwise.
- **Detects the language of the address:**
  If it's not English, the pipeline calls Google Translate to convert it. This handles records in Chinese, Arabic, Russian, Korean, Japanese, Hindi, Spanish, German, Italian, French, etc.
- **Normalises common abbreviations:**
  "St" becomes "Street", "Blvd" becomes "Boulevard", "Av." becomes "Avenida" for Spanish records, and so on.
- **Flags**, garbage streets, things like "VAT12345", "NA", "-", "goods receipt", or single generic words. These streets get dropped before going to the validation API, while the city and country are kept.

2. Stage 2 - Address Validation (Azure Maps)

Once the address is cleaned up, the pipeline sends it to Azure Maps -Microsoft's mapping API. Azure tries to find the exact address and returns a confidence score from 0 to 1, plus the type of match ("Point Address" means a specific building, "Street" means just the street, "Address Range" means a number block, etc.).

If the score is at least 0.85 AND the match is to a specific building (Point Address or Address Range), we treat the address as valid. If not, we don't give up, we try three fallback strategies in order:

1. Drop the street, search using just city + state + zip + country.
2. Drop the state and zip, search using street + city + country only.
3. Last resort: just city + country.

If all three fallbacks fail, the record is flagged for manual review at this stage.

> 96.8% of records (35,074 out of 36,230) passed address validation. Only 3.2% had addresses so broken that even the fallback chain couldn't help. The pipeline used the primary search for 67.5% of records, the rest needed at least one fallback.

3. **Stage 3 - Routing Decision**

Now we have a record with an address that's been validated to some degree. The pipeline picks one of three paths:

- **FULL_MATCH:**

  The address passed at high confidence and Azure pinned a specific building. This is the cleanest case (40.2% of records, 14,570). These go to Tier 1 first.

- **NAME_FIRST:**

  The address is too weak to anchor the search, but we have a usable company name and at least some location info (city or country). The pipeline searches by company name first and lets the AI find where the company actually is. This is most records (59.8%, 21,660) because international and partial-address records dominate.

- **SKIP:**

  Neither the address nor the name is usable. No automated work is possible. (0% in this run, every record had something to work with).

4. **Stage 4 - Company Match + Enrichment**

This is the heart of the pipeline. It tries to confirm that the company actually exists at the validated address, and if so, it pulls firmographic data about that company from authoritative web sources. There are three tiers, tried in order:

a. **Tier 1 - Azure Fuzzy POI Search**

   Azure has a "point of interest" search -given a company name and a location, it returns matching businesses with a similarity score. If the name similarity is at least 0.72 (or the country-specific threshold) AND the returned address matches the MDM address, this counts as a HIGH confidence match. We mark the record COMPANY_FOUND and move on.

   Tier 1 caught 3,894 records (10.7% of all records) in this run.

b. **Tier 2 - Azure POI Search (relaxed)**

   If Tier 1 doesn't match, we try Azure's broader POI search. This searches by company name with a wider radius. If we find a name match but the address verification is weaker, we mark the record COMPANY_FOUND with MEDIUM confidence (score 0.85).

   Tier 2 caught 1,348 records (3.7%).

c. **Tier 3 - OpenAI GPT-4.1 with Web Search**

   If Azure can't find the company at all, we hand the record to GPT-4.1 with the web_search_preview tool. The model is given the MDM record, the validated address, and instructions to perform 5 to 7 specific web searches: search for the company name in the city, look up its official website, check what business is at the MDM address, try translated/transliterated name variants for non-English records, and look up NAICS/SIC codes from US Census or SEC filings.

- The AI returns a structured JSON answer:

  Does the company exist? What's its canonical name? Is it actually at the MDM address, somewhere else, or unknown? What's its NAICS code, SIC code, parent company, employee count, revenue range, year founded, industry?

- The AI is also forced to cite the actual URLs it grounded its answer on.

  These get logged to columns called sources_verify, sources_enrich, and sources_hq. Anyone reviewing the output can click those URLs and check the AI's claim against the live web page.

- Tier 3 handled 30,988 records combined (85.5%) - most records needed AI help because Azure's POI database is sparse outside the US.

Headquarters Resolution (after the match succeeds)

Once a company is verified, the pipeline does one more step:

It tries to find the corporate headquarters. The logic is: if the AI flagged this MDM address as the company's HQ already, use it.

Otherwise, climb the corporate hierarchy, global ultimate, then domestic ultimate, then parent company, then the company itself, and run a separate AI search to find that entity's HQ address.

The result is cached so two records belonging to the same parent company don't pay for two AI calls.

C. What Challenges Did the Pipeline Have to Solve?

This wasn't "call an API and done." The data has 8 distinct quality problems that all needed solutions:

Challenge 1 - Encoding corruption (mojibake)

21.3% of records had street addresses where UTF-8 characters were corrupted into garbage. Solution: detect the canonical lead-byte + continuation-byte pattern, re-decode using Latin-1 or cp1252, and validate that the result is cleaner than the input. Affected records: 7,720.

Challenge 2 - Multilingual data

Records arrive in 50+ languages. A Spanish address with "Calle Mayor 23" or a Chinese name in Han characters won't match an English-only search. Solution: language detection on every record, automatic translation for non-English fields, plus prompt instructions to the AI to search both the original and translated forms.

Challenge 3 - Address quality varies wildly

Some addresses are perfect, some have missing street numbers, some have just a city, some have placeholders like "goods receipt loading dock". Solution: a 3-tier fallback chain that progressively drops the worst part of the address (street → state/zip → city only) before giving up.

Challenge 4 - Company names contain noise

Source names often include codes ("Tj Maxx 01607"), accounting tags ("Run Rate", "YTD"), or department labels ("Accounts Payable"). Solution: a clean_company() function that strips numeric suffixes, accounting jargon, and procurement labels before searching.

Challenge 5 - Hallucination risk in AI

GPT-4.1 might invent a NAICS code, a website, or claim a company is at an address it never actually verified. Five separate guardrails address this, see the next section.

Challenge 6 - Throughput

Sequential processing of 36K records would take days. Solution: 15 parallel workers, thread-local HTTP connection pooling, and a dedicated writer thread to avoid lock contention. End-to-end runtime: 4.5 hours.

Challenge 7 - Reproducibility and audit

If the AI is wrong, we need to know why. Every AI call records the URLs it grounded against, so any claim is independently verifiable.

D. How Is the Enrichment Verified? - 5 Built-in Mechanisms

The pipeline has five distinct verification and correction mechanisms baked in:

1. Mechanism 1 - Verdict Rigor Guard (Pipeline Downgrade)

This catches the AI's most common failure mode. The model sometimes claims address_match=CORRECT based purely on state-level or city-level overlap, without actually finding the specific street in the company's known locations.

The pipeline checks:

If the model says CORRECT but the MDM street number doesn't appear anywhere in the AI's all_locations text, the verdict is automatically downgraded to PARTIAL with LOW confidence. An evidence note is added explaining why.

In this run, 8,026 verdicts were auto-corrected by this rule. That's 8,026 cases where the AI overclaimed and the pipeline caught it.

2. Mechanism 2 - NAICS / SIC Format Validation

NAICS codes from US Census are exactly 2 to 6 digits. SIC codes are exactly 4 digits. After every AI call, a regex check runs: if the AI returned anything that doesn't match the format, the code is set to null and the description is cleared. This prevents fabricated industry codes from making it into the master record.

3. Mechanism 3 - Multi-Tier Deterministic-First Fallback

AI is the last resort, not the first. Tier 1 and Tier 2 both use Azure Maps, a deterministic, vendor-curated database. The AI in Tier 3 only runs when both Azure searches fail. This shrinks the surface area where hallucination is even possible, and for 14.4% of records (Tier 1 + Tier 2 hits) we never use AI at all.

4. Mechanism 4 - Source URL Citations

Every AI call uses GPT-4.1's web_search_preview tool, which forces the model to ground its answer in real web pages and emit URL citations. The pipeline extracts those URLs and writes them to three columns in the output:

- sources_verify - URLs used during company verification
- sources_enrich - URLs used during firmographic enrichment
- sources_hq - URLs used during headquarters lookup

An auditor can click any of these URLs and independently verify what the AI claimed.

5. Mechanism 5 - Address Gate + 3 - Fallback Chain

Before a record can take the FULL_MATCH path (Tier 1/2 with high-confidence verification), it must pass two checks: Azure score ≥ 0.85, AND match type must be "Point Address" or "Address Range" (not just "Street"). This second check matters, at score 0.93, Azure might match the street name without the building number. The gate prevents that from being treated as a verified match.

E. The Final Numbers

This is the result of running the full pipeline on all 36,230 records:

Item	Number
Total records processed	36,230
Address validation pass rate	96.8% (35,074 records)
Records requiring fallback correction	32.5% (used at least one of the 3 fallbacks)
Mojibake (encoding corruption) fixes	21.3% (7,720 records)
Routing to FULL_MATCH path	40.2% (14,570 records)
Routing to NAME_FIRST path	59.8% (21,660 records)
Tier 1 Azure Fuzzy match	10.7% (3,894 records)
Tier 2 Azure POI match	3.7% (1,348 records)
Tier 3 AI verification (FULL_MATCH path)	25.7% (9,328 records)
Tier 3 AI name-first verification	59.8% (21,660 records)
Pipeline downgrades (verdict rigor guard)	8,026 verdicts auto-corrected
Total verified (company_exists=True)	75.2% (27,261 records)
Fully enriched (all 6 core fields)	55.1% (19,954 records)
Partially enriched	18.7% (6,791 records)
Verified but no enrichment fields filled	1.4% (516 records)
Not verified	24.8% (8,969 records)
End-to-end runtime	~4.5 hours (15 parallel workers)

3716/8515 - Truly stuck = 1394/3716 = openai parsing error

F. The Manual Review Queue Story

After the pipeline finishes, every record gets one final classification: either it's done, or it's flagged for manual review. The original review filter caught 17,902 records (49.4% of all records), which felt high, almost half the dataset needing human eyes.

Investigating those 17,902 records revealed that many were flagged due to overly strict filter logic, not because the records actually needed human attention. We wrote a separate cleanup script that applies four targeted fixes to the manual review sheet:

Fix A - UNVERIFIED → COMPANY_FOUND_DIFF_ADDR (935 records)

These records had company_exists=True (the AI confirmed the company exists) but address_match=UNKNOWN (it couldn't pin the specific street). The pipeline code mapped these to UNVERIFIED, which triggered the review filter. But UNKNOWN doesn't mean the company is unverified, it just means the address couldn't be pinned. We relabel these to COMPANY_FOUND_DIFF_ADDR and remove them from the queue. Safety guard: only applied where confidence_score >= 0.65.

Fix B - DIFF_ADDR with verified best_address (7,345 records)

These records had match_status=COMPANY_FOUND_DIFF_ADDR meaning the company is verified but at a different address than what's in MDM. The pipeline already found and stored the correct address in the best_address column. No human needs to re-look these up. We add a corrected_address column with the verified address and remove the records from the queue. Safety guard: best_address must be at least 10 characters long (filters out 42 truncated values).

Fix C - COMPANY_FOUND over-caught (19 records)

Records with match_status=COMPANY_FOUND and ai_confidence=HIGH but caught by the review filter because of a secondary tag (an addr_final_status error or a pipeline downgrade note). These are fully verified, the secondary tag is irrelevant once the company match has succeeded at HIGH confidence. We remove them from the queue.

Fix D - NOT_FOUND with high-confidence best_address (1,088 records)

These are records where the AI was HIGH or MEDIUM confident the company is NOT at the MDM address, but it found where the company actually is. The company is real -the MDM address is just wrong. Functionally identical to DIFF_ADDR. We promote these to COMPANY_FOUND_DIFF_ADDR and copy best_address to corrected_address.

> **Result of the cleanup:**
>
> Manual review queue went from 17,902 records (49.4% of total) down to 8,515 records (23.5% of total). 9,387 records were removed by the four fixes. Every change is logged in an audit_log.csv with before/after values, the reason, and the confidence score, so any change can be reviewed and reversed if needed.

G. What's Still in the Review Queue & What to Do Next

The 8,515 records still in review break into four buckets, each with a different recommended action:

3,716 truly stuck (43.6% of remaining)

Small/local businesses with no web presence, dissolved companies, or generic code-like names like "TLSI" or "RMWB".

D&B couldn't enrich these either -that's why they're in this dataset. Recommendation: accept as UNVERIFIABLE in MDM, or route to D&B Match API as a paid lookup.

The mdm_address_occupant column already has whatever Azure POI reverse-geocoded at the physical address, which is the best available answer.

2,030 UNVERIFIED + LOW confidence (23.8% of remaining)

AI returned LOW confidence and didn't commit. Most are international records. **Recommendation: t**argeted re-run with a prompt enhancement, add explicit instructions to search in local language for CN, ES, IT, MX, DE records, and pass the original MDM name_reason and name_sub_category fields as extra context.

2,393 NOT_FOUND with best_address (28.1% of remaining)

649 are pipeline downgrade victims, fixable with a code change to _enforce_address_match_rigor() that loosens the street-number check for non-US countries. The remaining 1,744 need a re-run with the international prompt.

376 other / ambiguous (4.4% of remaining)

DIFF_ADDR records without a confirmed best_address, address validation errors, and remaining UNVERIFIED cached records. These need manual triage.

The irreducible floor, records no automated pipeline change can ever resolve -is roughly 3,716 records, or about 10.3% of the original 36,230 dataset. Everything above that floor is fixable with code changes or targeted re-runs.

H. Did We Achieve the Original Goal?

The original project goal was: take the records that D&B couldn't enrich and recover them. Specifically, validate the address, verify the company, and enrich with legal name, hierarchy, NAICS/SIC, website, email, HQ.

Scorecard:

- **Validate & correct addresses:**

  Achieved. 96.8% pass rate using Azure Maps with a 3-fallback chain.
- **Verify company presence:**

  Achieved. 27,261 records (75.2%) have company_exists confirmed via deterministic API or AI web search with cited sources.
- **Enrich legal name, NAICS, SIC, website, email domain, HQ:**

  Achieved. 90% legal name fill, 78% NAICS, 96% HQ resolution on verified records.
- **Handle malformed / multilingual / mojibake records:**

  Achieved. 21.3% mojibake auto-rescued, language detection + translation built in for 50+ languages.
- **Enrich parent / domestic / global ultimate hierarchy:**

  Partial. Only 23 to 34% filled. **Reason:** most records in this dataset are standalone SMBs with no public corporate hierarchy data anywhere on the web. This is a data availability problem, not a pipeline problem -D&B faces the same gap.
- **Reduce manual review burden:**

  Partial. From 49.4% to 23.5% via four automated fixes, with ~3,716 records that no automation can ever resolve.

> **Bottom line:**
> 55.1% of the original dataset is fully enriched and ready for MDM today.
> Another 18.7% is verified with most fields filled in. 23.5% needs review, of which roughly half is fixable with one more targeted re-run.
> About 10% -the small/local/defunct businesses -will never be enrichable by any automated process and need either acceptance as UNVERIFIABLE or a paid D&B Match lookup.

I. How to Run the Cleanup Script

To reproduce the manual review cleanup on your own machine:

1. Install Python 3.8 or higher.
2. Open a terminal and install dependencies: pip install pandas openpyxl
3. Place the script clean_manual_review_vscode.py in the same folder as mdmep_Final.xlsx.
4. Run: python clean_manual_review_vscode.py
5. The script reads the Manual Review sheet, applies the four fixes, prints a summary to the terminal, and creates two output files in the same folder:
    a. **mdmep_Final_cleaned.xlsx:**

       The updated workbook with four sheets:

       - Manual_Review_Full (all original records with fix tags)
       - Still_Needs_Review (the cleaned queue of 8,515 records)
       - Removed_From_Review (9,387 records that were removed, with corrected addresses for the MDM team)
       - Audit_Log (every change made).

    b. **audit_log.csv:**

       Same audit log as a CSV for easy filtering.

Nothing in the original Excel file is overwritten. The script only reads from the Manual Review sheet and writes a new file. Every change is documented in the audit log with the MDM_KEY, what column changed, the value before, the value after, and the reason.